

实验 1：深度学习基础/MLP

实验 1：深度学习基础/MLP

概述

环境配置

目录结构

实验任务

Task 1：标量自动微分

Task 2：张量自动微分

为什么要换一套实现

从分量偏导到矩阵形式

Element-wise 广播机制及其反传

全图回传 backward

softmax 与交叉熵应当合并为一个节点

Task 3：组装 MLP 并完成分类任务

训练数据集：圆环

MLP 网络结构

评估指标：IoU 而非准确率

训练循环

可视化

验收

参考资料

概述

深度学习（Deep Learning）是机器学习的一个分支，其特点是用多层结构逐级处理数据，让模型自己学会提取有用的特征[1]。从多层感知机（MLP）到卷积、循环神经网络（CNN/RNN），再到大语言模型（LLM）所依赖的 Transformer 架构，这一研究领域已经发展出了许多不同的模型结构。不同结构的适用场景不同，但它们的训练方式却高度一致：定义一个衡量预测好坏的**损失函数（Loss Function）**，计算损失对每个参数的梯度，再沿梯度的反方向调整参数。这种参数更新方式称为**梯度下降（Gradient Descent）**，而梯度的计算则依靠**反向传播（Back propagation）**（参考[2]）。

本次实验不使用任何现成的深度学习框架，而是从零实现这套机制，并用它训练一个**多层感知机（MLP）**。主要包括三个任务：

1. **标量自动微分**：一个节点存一个数，用最透明的形式实现反向传播。
2. **张量自动微分引擎**：沿用 Task 1 的调度机制，把节点从一个数换成一个矩阵。这也是 PyTorch 等真实框架的组织方式。
3. **组装并训练 MLP**：用 Task 2 的引擎搭出全连接层并堆成 MLP，训练它判断平面上的一个点是否落在一个圆环内，最后把整张网格喂回网络逐点着色，重新画出这个圆环。

Task 1 和 Task 2 实现的都是**自动微分（Automatic Differentiation）**，结果在浮点精度内精确；它不同于用有限差分估计导数的**数值微分（Numerical Differentiation）**。本实验提供的 `src/gradient_check.py` 是数值微分工具，需要时可用于交叉校验。

本次实验不可使用 PyTorch 等自动微分工具，仅可使用源文件中导入的库（`numpy`、`math`）。

环境配置

请首先安装 [uv](#) Python 项目管理工具（后续实验应该也会用到）：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# On windows.
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

先确认安装完毕，然后在项目根目录下执行：

```
uv sync
```

该指令会在根目录下创建好 Python 虚拟环境。完成本实验所需的依赖仅存在于该虚拟环境中，若使用 IDE，可先将解释器设置为该环境中的解释器。

之后可以尝试运行测试用例（**路径中不可带非 ascii 编码字符**）：

```
uv run pytest
```

若输出类似以下的内容，则说明环境配置完毕：

```
FFFFFFFFFFFFFFFF.FFFFFFFFFF.FFFFFFFFFFFFFFFFFF [100%]
===== short test summary info =====
FAILED tests/test_engine.py::test_forward_basic_ops - NotImplementedError
FAILED tests/test_engine.py::test_backward_single_op - NotImplementedError
FAILED tests/test_engine.py::test_backward_accumulates_gradients - NotImplementedError
...
40 failed, 2 passed in 4.73s
```

默认配置下 pytest 不打印调用栈（`--tb=no`），只给出通过/失败的清单。调试时可以加上 `--tb=short` 看具体报错，或者用 `uv run pytest [文件路径]::[用例名]` 单独跑某一个用例。

目录结构

```
lab1/
├── src/
│   ├── engine.py          # Task 1: 标量自动微分
│   ├── tensor.py          # Task 2: 张量自动微分
│   ├── nn.py              # Task 3: 全连接层、MLP、训练循环
│   ├── data.py            # Task 3: 数据集构造与评估指标
│   └── gradient_check.py  # 数值梯度校验工具（已实现，请勿修改）
├── tests/                  # 测试用例（请勿修改）
├── visualize.py           # 可视化脚本
└── pyproject.toml
```

实验任务

Task 1: 标量自动微分

反向传播的本质是链式法则在**计算图 (Computational Graph)** 上的自动化。把一个复杂表达式拆解成基本运算，运算之间的依赖关系构成一张有向无环图；每个基本运算只需知道自己的局部导数，从最终输出出发沿图反向走一遍，把局部导数逐层相乘并累加，就能得到输出对图中任意节点的偏导数。该算法由 Rumelhart 等人于 1986 年正式提出[3]，至今仍是所有深度学习框架的核心。详细的讲解可以参考[4]。

实现时注意以下细节：

1. **梯度必须累加而非覆盖**：当一个中间结果被下游使用不止一次时，它的梯度是所有路径贡献之和。
2. **反向传播需按拓扑序进行**：只有当一个节点的所有下游贡献都累加完毕后，才能用它的梯度去更新父节点。

下面请在 `src/engine.py` 中补全 `Value` 类的方法。该类包裹一个标量，重载算术运算符以在计算过程中动态构建计算图；调用 `.backward()` 后，图中所有节点的 `.grad` 属性应被正确填充。`__add__` 已经写好，作为其余运算的范例。

请你完成以下任务：

- ☐ 实现 `engine.py::Value.__mul__`：乘法。
- ☐ 实现 `engine.py::Value.__pow__`：幂运算，指数为常数（不是 `Value`）。
- ☐ 实现 `engine.py::Value.relu`： $\max(0, x)$ ，**零点处的导数按惯例取 0**。
- ☐ 实现 `engine.py::Value.tanh`： $\tanh(x)$ 。
- ☐ 实现 `engine.py::Value.exp`：自然指数 e^x 。
- ☐ 实现 `engine.py::Value.log`：自然对数 $\ln x$ 。
- ☐ 实现 `engine.py::Value.backward`：把当前节点视为最终输出，填充其所到之处全图节点的 `.grad`。**注意梯度必须累加，以及回传的顺序**（tip：先做拓扑排序，再逆序调用各节点的 `_backward`）。

完成后应该能通过的测例 (10)：`tests/test_engine.py` 下的全部单测。

Task 2: 张量自动微分

为什么要换一套实现

Task 1 中一次矩阵乘法会产生成千上万个节点，每个都带一个闭包和一个集合，开销在真实模型上的开销无法接受。真实框架的做法是让**图节点的粒度是张量运算而非标量运算**：如在 PyTorch 中 `A @ B` 是单个节点，反向时直接做两次矩阵乘法得到两个操作数的梯度。

这不是另一套数学原理，而是同一个链式法则的向量化写法。

本任务需要补全 `src/tensor.py`。其中的 `Tensor` 类与 `Value` 结构完全一致：同样有 `_prev`、`_backward`、`backward()`，只是 `data` 和 `grad` 从浮点数变成了 NumPy 数组。

从分量偏导到矩阵形式

以线性层 $S = XW + b$ 为例 (X 形状为 (N, D) , 是含 N 个 D 维样本的一个批次; W 形状为 (D, C) ; b 形状为 $(C,)$), 先写出分量形式的偏导:

$$S_{ic} = \sum_j X_{ij} W_{jc} + b_c \implies \frac{\partial S_{ic}}{\partial W_{jc}} = X_{ij}$$

代入链式法则得到

$$\frac{\partial L}{\partial W_{jc}} = \sum_i \frac{\partial L}{\partial S_{ic}} \frac{\partial S_{ic}}{\partial W_{jc}} = \sum_i X_{ij} \frac{\partial L}{\partial S_{ic}}$$

把这个对下标 i 的求和识别为矩阵乘法, 即得

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial S}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial S} W^\top$$

那个求和号, 正是标量图中「一个节点被多条路径使用、梯度需要累加」这件事的向量化形式。所以 $\frac{\partial L}{\partial W}$ 和 $\frac{\partial L}{\partial X}$ 公式的矩阵乘法形式并不是新原理, 它们只是把 $N \times D \times C$ 次标量乘加打包成了一次矩阵乘。

关于矩阵微分的更多数学细节请参考[5]。

请你完成以下任务:

☐ 实现 `tensor.py::Tensor.__matmul__`: 矩阵乘法, 本实验中只需处理二维输入。

完成后应该能通过的测例 (1): `test_tensor.py::test_matmul_forward`。

Element-wise 广播机制及其反传

element-wise (逐元素) 运算是指对张量中每个位置独立施加的运算。单目运算 (如 `exp()`、`relu()`) 天然如此; 双目运算中, `+`、`-`、`*`、`/` 等也是逐元素的——取两个张量对应位置上的元素做一次运算, 得到结果对应位置上的元素。而矩阵乘法 `@` 则不属于这一类, 它的输出元素由整行与整列的内积得到。

NumPy 允许形状不同的两个数组做 element-wise 运算, 规则如下:

1. 若两者维度数不同, 在**较短的那个 shape 左侧补 1**, 直到维度数相同;
2. 逐维比较, **同一位置的维度要么相同, 要么其中一个为 1**, 否则报错;
3. 长度为 1 的那一维会被拉伸到与另一方相同的长度。

举几个例子:

$(N, C) + (C,)$	$\rightarrow (C,) \text{ 补成 } (1, C), \text{ 第 } 0 \text{ 维 } 1 \rightarrow N$	$\rightarrow (N, C)$
$(3, 1) + (1, 4)$	$\rightarrow \text{第 } 0 \text{ 维 } 1 \rightarrow 3, \text{ 第 } 1 \text{ 维 } 1 \rightarrow 4$	$\rightarrow (3, 4)$
$(2, 5, 3) + (5, 3)$	$\rightarrow \text{第 } 2 \text{ 个补成 } (1, 5, 3), \text{ 第 } 0 \text{ 维 } 1 \rightarrow 2$	$\rightarrow (2, 5, 3)$
$(3, 4) + (2, 4)$	$\rightarrow \text{第 } 0 \text{ 维 } 3 \text{ 与 } 2 \text{ 既不相等也不含 } 1$	$\rightarrow \text{报错}$

现在考虑反向。前向被拉伸出来的每一份拷贝, 在反向时都会各自收到一份梯度贡献; 而参数本身只有一个, 所以这些贡献必须累加回去。**累加后, 得到的梯度形状应该和操作数的形状相同。**

请你完成以下任务:

☐ 实现 `tensor.py::_unbroadcast(grad, shape)`: 输入 `grad` 是广播后形状的梯度, 输出必须是形状恰好为

`shape` 的数组。该函数的设定是通用实现，它应当能处理 NumPy 允许的任意一对形状，而不只是 $(C,) \rightarrow (N, C)$ 这一种。

- ☐ 实现 `tensor.py::Tensor.__mul__`：逐元素乘法。提示：写法可以参考已给出的逐元素加法 `__add__`，且应当复用 `_unbroadcast`。
- ☐ 实现 `tensor.py::Tensor.relu`：逐元素 $\max(0, x)$ ，零点处的导数按惯例取 0。

完成后应该能通过的测例 (6)：

```
test_tensor.py::test_unbroadcast_removes_prepended_axes、
test_tensor.py::test_unbroadcast_keeps_length_one_axes、
test_tensor.py::test_unbroadcast_mixed_case、
test_tensor.py::test_unbroadcast_is_identity_on_matching_shape、
test_tensor.py::test_mul_forward_and_broadcast、
test_tensor.py::test_relu_forward。
```

补充说明：

- `_unbroadcast` 是与算子无关的：它只处理形状，不关心梯度是怎么算出来的。因此所有逐元素双目运算（加、减、乘、除、幂等）都能共用同一个实现。本任务要求你实现 `__add__` 和 `__mul__` 两个算子，请思考如何复用 `_unbroadcast`。
- 事实上，矩阵乘法 `@` 也支持广播，但规则不同：它把最后两维当作矩阵参与乘法，只对前面的批量维度按上述逐元素规则广播。本实验默认 `__matmul__` 只处理二维输入，不涉及这种情况。

全图回传 backward

至此各个算子都已就位，接下来我们把它们串起来进行反向传播。这一部分与 Task 1 中 `value.backward` 的做法几乎完全相同，唯一的区别是起始梯度不再是一个标量，而是和输出同样形状的数组。

请你完成以下任务：

- ☐ 实现 `tensor.py::Tensor.backward`：把当前节点视为最终输出，填充图中所有节点的 `.grad`。

完成后应该能通过的测例 (9)：

```
test_tensor.py::test_add_backward_sums_over_broadcast_axis、
test_tensor.py::test_add_backward_general_broadcast、
test_tensor.py::test_mul_backward、
test_tensor.py::test_mul_and_add_share_unbroadcast、
test_tensor.py::test_matmul_backward、
test_tensor.py::test_relu_backward、
test_tensor.py::test_gradient_accumulates_when_reused、
test_tensor.py::test_topological_order_is_respected、
test_tensor.py::test_agrees_with_scalar_engine。
```

softmax 与交叉熵应当合并为一个节点

分类模型的输出层需要做两件事：把一组实数分数转换为概率分布，再衡量这个分布与真实答案的差距。这两件事分别由 softmax 和交叉熵完成。

softmax 把一个长度为 C 的实数向量映射成一个概率分布（各分量非负且和为 1）：

$$p_c = \frac{e^{s_c}}{\sum_k e^{s_k}}$$

其中 s 是模型输出的原始分数，未经归一化，通常称为 **logits**。取指数保证了结果为正，除以总和保证了归一化；同时指数会放大分数之间的差距，分数最高的那一类会拿到明显更大的概率。

交叉熵 (Cross Entropy) 衡量两个概率分布的差距。设真实分布为 q 、预测分布为 p ，其定义为

$$H(q, p) = - \sum_c q_c \log p_c$$

在分类任务中真实分布是确定的（正确类别概率为 1，其余为 0），即真实标签的 **one-hot 编码**。这使得求和中只剩下正确类别那一项，交叉熵退化为 $-\log p_t$ (t 为正确类别)。

把两者组合起来，一批 N 个样本、 C 个类别的损失即为：

$$p_{ic} = \frac{e^{s_{ic}}}{\sum_k e^{s_{ik}}}, \quad L = -\frac{1}{N} \sum_i \log p_{i, y_i}$$

其中 s_{ic} 是第 i 个样本属于第 c 类的分数， y_i 是该样本正确类别的下标。这一组合是分类模型输出层的标准配置，它并非随意选择，而是类别分布下最大似然估计的直接结果，若对其概率解释感兴趣可参考[6]。

把两步合并成一个计算图节点有两个理由：

- **数值稳定性**：朴素地计算 e^s 在 s 较大时会溢出为 `inf`，而训练中 logits 达到这个量级并不罕见。标准解法是先减去每行的最大值 $m_i = \max_k s_{ik}$ （分子分母同乘一个常数结果不变，但所有指数的输入都不大于 0，从而避免上溢）。进一步地，直接计算 $\log p$ （即 `log_softmax`）而不是先算 p 再取对数，可以避免「先 exp 再 log」的精度损失。
- **梯度形式极简**：合并之后 $\partial L / \partial s$ 可以化简成一个非常简洁的形式，请自行推导。提示：先从 $N = 1$ 的情形入手，把 L 写成只含 s 的表达式再求偏导。

事实上 PyTorch 的 `torch.nn.CrossEntropyLoss` 就是这么做的，原因完全相同。

请你完成以下任务：

- ☐ 实现 `tensor.py::softmax_cross_entropy(logits, y)`：融合的 `log_softmax` + 交叉熵算子，返回一个标量 `Tensor`。务必按上文所述做数值稳定化处理，否则大 logits 下会溢出为 `inf`。

完成后应该能通过的测例 (5)：

```
test_tensor.py::test_cross_entropy_uniform_and_confident、
test_tensor.py::test_cross_entropy_numerical_stability、
test_tensor.py::test_log_softmax_avoids_underflow、
test_tensor.py::test_cross_entropy_gradient、
test_tensor.py::test_full_graph_gradient。
```

综上，整个 Task 2 共涉及 22 个测例，全部位于 `tests/test_tensor.py`；其中 `test_add_forward_and_broadcast` 对应已给出的 `__add__`，无需实现即可通过。

Task 3: 组装 MLP 并完成分类任务

有了张量引擎，剩下的工作就是把它组织成网络并训练。

训练数据集：圆环

在 $[-1, 1] \times [-1, 1]$ 的方形区域上铺一张 $n \times n$ 的网格，每个网格点的坐标 (x, y) 就是一个输入样本；若该点到原点的距离落在 $[r_{\text{inner}}, r_{\text{outer}}]$ 之间则标签为 1，否则为 0。

两个设计细节值得说明：

- **坐标必须归一化到 $[-1, 1]$** ，而不是直接用 $0 \dots n - 1$ 的网格下标。后者输入尺度大且不居中，会让梯度下降收敛得非常慢。
- **选圆环而不是圆**，是因为圆环不仅线性不可分，而且要求网络同时学出内外两条边界，比单一闭合边界更能体现 MLP 的表达能力。

请你完成以下任务：

- ☐ 实现 `data.py::make_ring(n, r_inner, r_outer)`：在 $[-1, 1] \times [-1, 1]$ 上铺 $n \times n$ 的网格，返回形状 $(n^2, 2)$ 的坐标数组与长度 n^2 的整数标签数组。

完成后应该能通过的测例 (1)：`test_nn.py::test_ring_dataset`。

MLP 网络结构

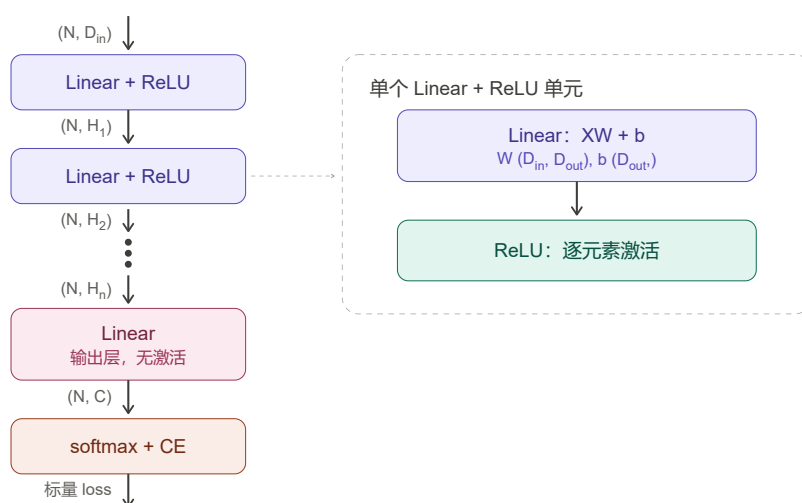
多层感知机 (Multi-Layer Perceptron, MLP) 又称前馈神经网络 (Feedforward Neural Network, FNN) 是最基础的一类神经网络，结构很简单：输入向量交给一层层运算依次处理，每一层先做一次线性变换，再套上一个非线性的激活函数，最后一层的输出就是模型的预测。信号自始至终单向流动，不回头也不循环，因此又称前馈神经网络 (Feedforward Neural Network, FNN)。

其中激活函数是关键。若把它去掉，多层线性变换叠起来仍然等价于一个线性变换，模型就只能画直线；而加上激活函数则可引入非线性，层数越多能表达的形状就越复杂（要拟合本任务的圆环，就必须要借助激活函数）。

MLP 要求输入是定长向量，且不假设输入具有空间或时序结构（这也正是其与 CNN/RNN 的本质区别），因此常用于表格数据的分类与回归任务。

MLP 也是更复杂结构的基本构件。Transformer 的每个 block 中，注意力负责在不同位置之间交换信息，其后紧跟的前馈层 (FFN) 则是一个双层 MLP，并且往往占据了模型的大部分参数量。

MLP 的核心组件是**全连接层 (Fully Connected Layer)**，一般也叫做**线性层 (Linear)**，持有两个参数 W (形状 $(D_{\text{in}}, D_{\text{out}})$) 和 b (形状 $(D_{\text{out}},)$)，前向即 $XW + b$ 。MLP 把若干个全连接层堆起来，层与层之间插入不带权重的 ReLU 激活层；最后一层不加激活，直接输出 logits 交给 `softmax_cross_entropy`。其结构示意图如下：



参数初始化需要注意：**权重不能初始化为全零**。本实验采用 **Kaiming (He) 初始化**：从标准正态分布采样后乘以 $\sqrt{2/D_{in}}$ ，这个缩放系数能让信号在多层 ReLU 网络中传播时方差保持稳定。

请你完成以下任务：

- ☐ 实现 `nn.py::Linear.__init__`：创建 `self.W` (形状 (D_{in}, D_{out})) 与 `self.b` (形状 $(D_{out},)$) 两个 Tensor。**权重按 Kaiming 初始化，即标准正态采样后乘以 $\sqrt{2/D_{in}}$ ；偏置初始化为零。**请使用传入的 `rng` 采样，以保证结果可复现。
- ☐ 实现 `nn.py::Linear.__call__`：前向计算 $XW + b$ 。
- ☐ 实现 `nn.py::MLP.__init__`：按 `sizes` 中相邻两项的宽度依次构造 `Linear` 层，存入 `self.layers`。
- ☐ 实现 `nn.py::MLP.__call__`：逐层前向，层与层之间插入 ReLU，**最后一层不加激活**，直接返回 logits。

完成后应该能通过的测例 (4)：

```
test_nn.py::test_linear_shapes_and_bias_init、
test_nn.py::test_linear_weight_scale、
test_nn.py::test_mlp_shapes_and_parameters、
test_nn.py::test_mlp_hidden_layers_are_activated。
```

评估指标：IoU 而非准确率

圆环大约只占整个网格的五分之一，因此一个「全部预测为 0」的退化模型就能拿到约 80% 的准确率。本实验改用**交并比 (IoU, Intersection over Union)**：

$$\text{IoU} = \frac{|\{\hat{y} = 1\} \cap \{y = 1\}|}{|\{\hat{y} = 1\} \cup \{y = 1\}|}$$

它只奖励与圆环本身的重叠，上述退化解的 IoU 为 0。该指标已在 `data.py::iou` 中实现，不需要修改。

训练循环

一个 epoch 的流程是：清空所有参数的梯度 → 前向 → 计算损失 → `loss.backward()` → 沿梯度反方向更新每个参数。本实验采用**全批量 (full-batch)** 梯度下降，即每个 epoch 用上全部样本。写出来大致是这样：

```
for epoch in range(num_epochs):
    for p in params:
        p.zero_grad()
    logits = model(inputs)
    loss = softmax_cross_entropy(logits, y)
    loss.backward()
    for p in params:
        p.data -= lr * p.grad
```

如果你写过 PyTorch，应该会觉得这段代码很眼熟，它和真实的训练循环几乎逐行对应。

请你完成以下任务：

- ☐ 实现 `nn.py::predict(model, x)`：前向一次，取 logits 在类别维上的 `argmax` 作为预测类别。
- ☐ 实现 `nn.py::train(model, x, y, lr, num_epochs, log_every)`：全批量梯度下降，返回损失历史。**每个 epoch 开始前须清空所有参数的梯度**，否则梯度会跨 epoch 累加。`history[t]` 记录的是第 t 次更新之前的损失，因此 `history[0]` 对应初始参数。

完成后应该能通过的测例（4）：

```
test_nn.py::test_train_history_and_progress、  
test_nn.py::test_train_updates_every_parameter、  
test_nn.py::test_mlp_fits_the_ring、  
test_nn.py::test_network_without_hidden_layer_fails。
```

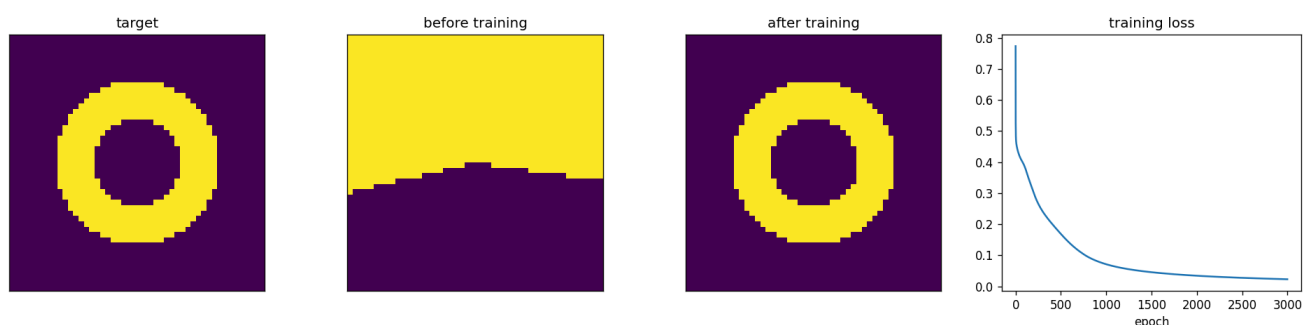
Task 3 共涉及 10 个测例，全部位于 `tests/test_nn.py`；其中 `test_iou_metric` 对应已给出的 `iou`，无需实现即可通过。

可视化

三个任务全部完成后，运行：

```
uv run python visualize.py
```

脚本会在 48×48 的网格上训练一个 `[2, 32, 32, 2]` 的网络，并输出 `ring.png`，从左到右四张图分别是：目标圆环、**未训练**网络的预测、**训练后**网络的预测、以及训练损失曲线。参考的运行结果如下：



参考运行结果的 IoU 从 0.196 上升到 0.998，第三张图与第一张图几乎无法区分。未训练的那张图很值得看一眼：它通常是一条折线把平面切成两半：这就是随机初始化的 ReLU 网络所能表达的形状。对比训练后的圆环，「学习」这件事在这里是肉眼可见的。

你也可以自行修改 `visualize.py` 中的网格分辨率、网络宽度、学习率和 epoch 数，观察它们对结果的影响。

验收

本次实验的分数由测试用例的通过比例决定，计算方式为 $\frac{\text{通过用例数}}{\text{总用例数}} \times 100\%$ ，共 42 个用例。

请按要求完成实验代码，之后：

1. 将根目录下的 `.venv`、`.pytest_cache` 目录以及 `ring.png` 删除（如有）；
2. 将根目录（`tab1`）以 zip 格式打包提交，压缩包命名为 `学号-姓名`。

注意以下几点：

- 实验过程中，除了 `uv` 自动生成的文件，你只能修改 `src` 下的源文件（`src/gradient_check.py` 除外）。
- 助教会以运行测试用例（`uv run pytest`）的结果为主要评分依据，但会对代码进行抽查。不得针对测试用例硬编码、或以其他方式绕过计算。

提交 ddl: 2026 年 9 月 30 日。

参考资料

- [1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press, Chapter 6: Deep Feedforward Networks. <https://www.deeplearningbook.org/>
- [3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
- [4] Karpathy, A., et al. Backpropagation, Intuitions. *CS231n: Deep Learning for Computer Vision*, Stanford University. <https://cs231n.github.io/optimization-2/>
- [5] Johnson, J. Derivatives, Backpropagation, and Vectorization. *CS231n Course Notes*, Stanford University. <https://cs231n.stanford.edu/handouts/derivatives.pdf>
- [6] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, §4.3.4. <https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>